



University
of Exeter

COMM113

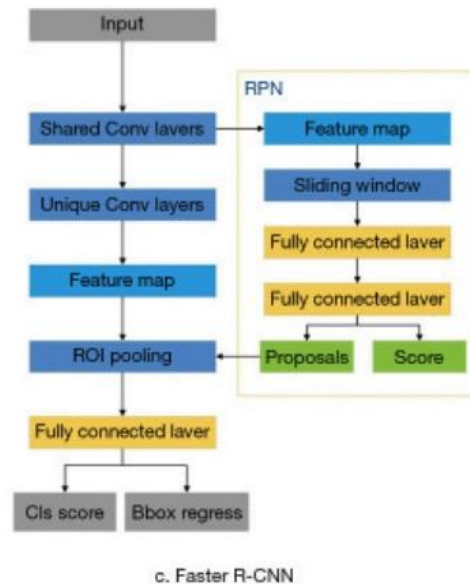
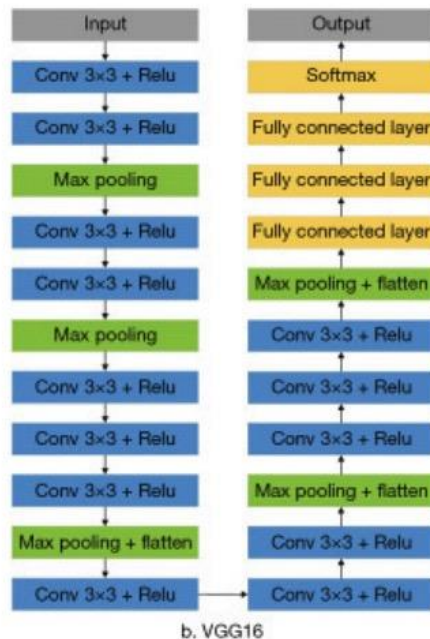
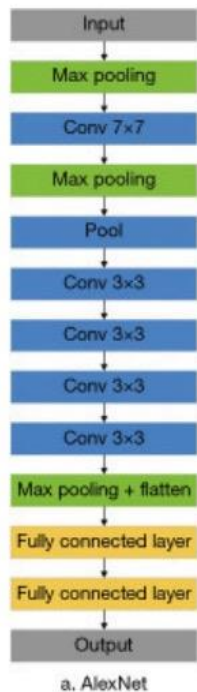
Deep Learning – Week 8

Recurrent Neural Networks

Gaojie (Jay) Jin
g.jin@exeter.ac.uk

Convolutional Neural Network

Typical Architectures

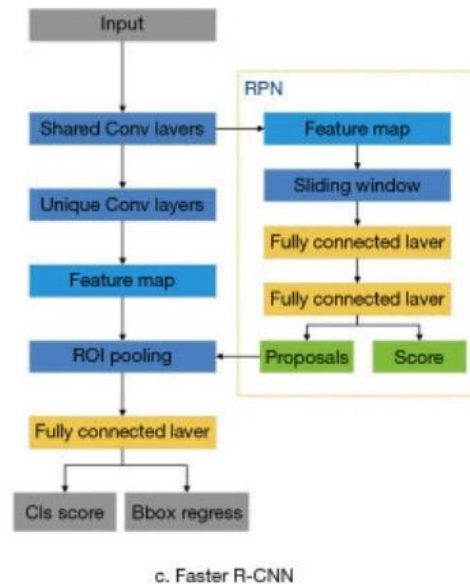
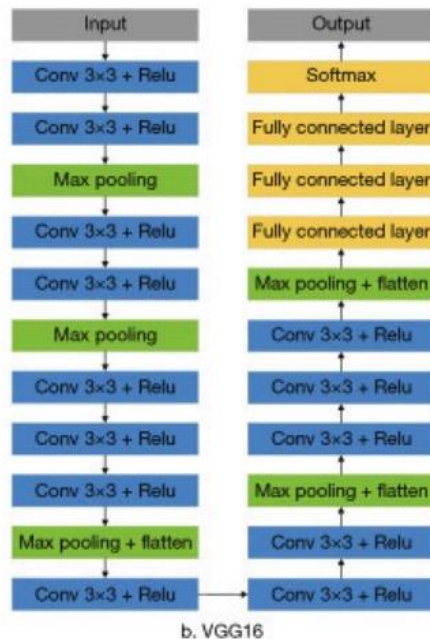
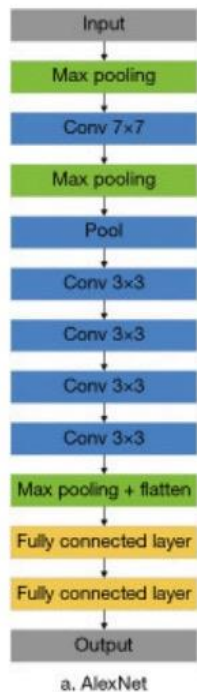


Q: Why are CNNs not ideal for sentence modelling?

CNNs are excellent for spatial data but not ideal for sequences.

Convolutional Neural Network

Typical Architectures



Q: Why are CNNs not ideal for sentence modelling?

A: They assume fixed-size input and lack explicit temporal modelling.

CNNs are excellent for spatial data but not ideal for sequences.

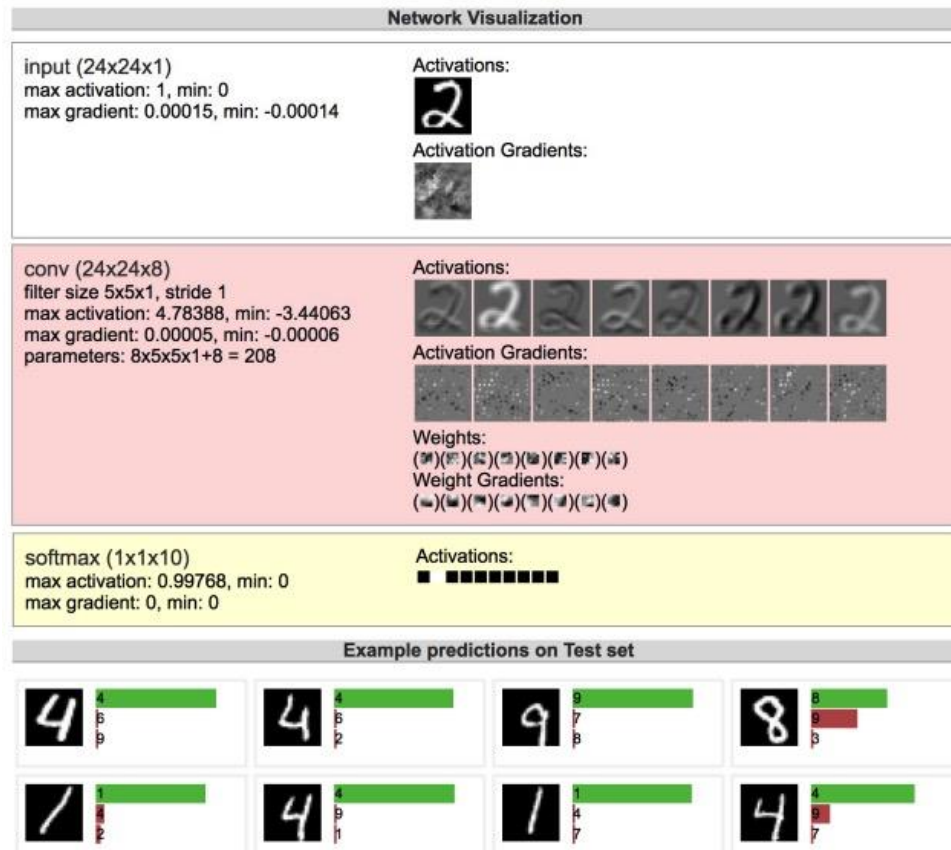
Visualisation of CNN

https://adamharley.com/nn_vis/cnn/2d.html



MNIST Digit Recognition with CNNs

<https://cs.stanford.edu/people/karpathy/convnetjs/demo/mnist.html>



CNN Summary

- **CNNs**
 - Used for almost all aspects of computer vision
 - Able learn spatial features
 - Typically, consist of convolution layers, pooling layers, nonlinearities, and fully connected layers

Q: What key assumption does CNN rely on for its success?

CNN Summary

- **CNNs**
 - Used for almost all aspects of computer vision
 - Able learn spatial features
 - Typically, consist of convolution layers, pooling layers, nonlinearities, and fully connected layers

Q: What key assumption does CNN rely on for its success?

A: Local spatial correlation (nearby pixels matter more).

CNN Summary

- **CNNs**
 - Used for almost all aspects of computer vision
 - Able learn spatial features
 - Typically, consist of convolution layers, pooling layers, nonlinearities, and fully connected layers

How to deal with time series data?

- **RNNs**
- **Transformers**

Word Embeddings

Key Idea:

- represent each word in your vocabulary as a vector
 - store as a $V \times D$ matrix
- where:
 V = number of words in vocab.
 D = vector's dimension

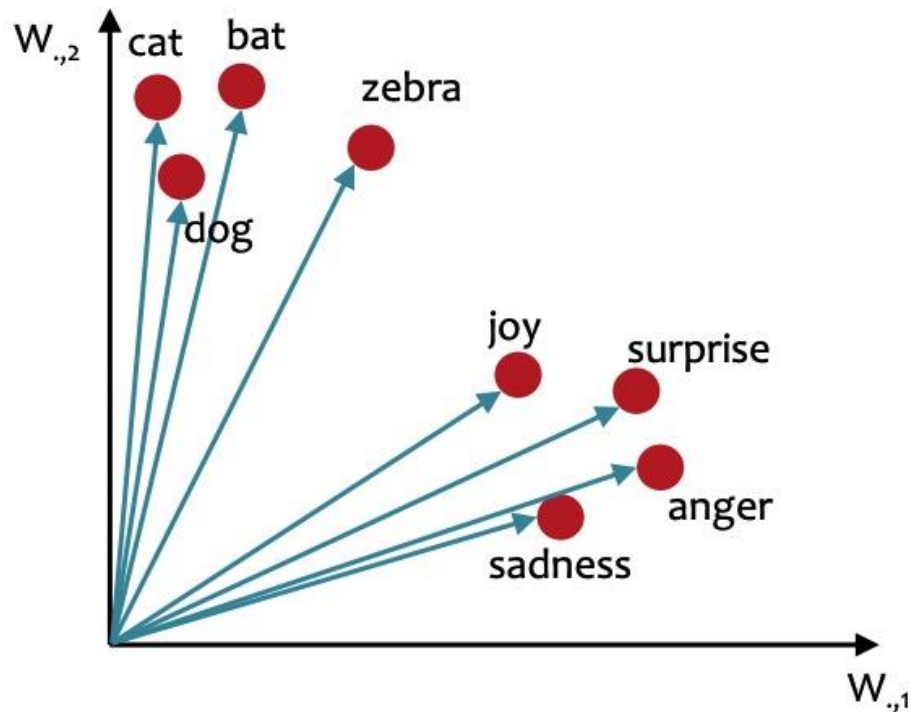
Modeling:

- define a model in which the vectors are parameters
- each copy of the word uses the same parameter vector
- train model so that similar words have high cosine similarity

anger
bat
cat
dog
joy
sadness
surprise
zebra

W

W_{11}	W_{12}
W_{21}	W_{22}
W_{31}	W_{32}
W_{41}	W_{42}
W_{51}	W_{52}
W_{61}	W_{62}
W_{71}	W_{72}
W_{81}	W_{82}



Word Embeddings

Q: Why not use one-hot vectors?

Key Idea:

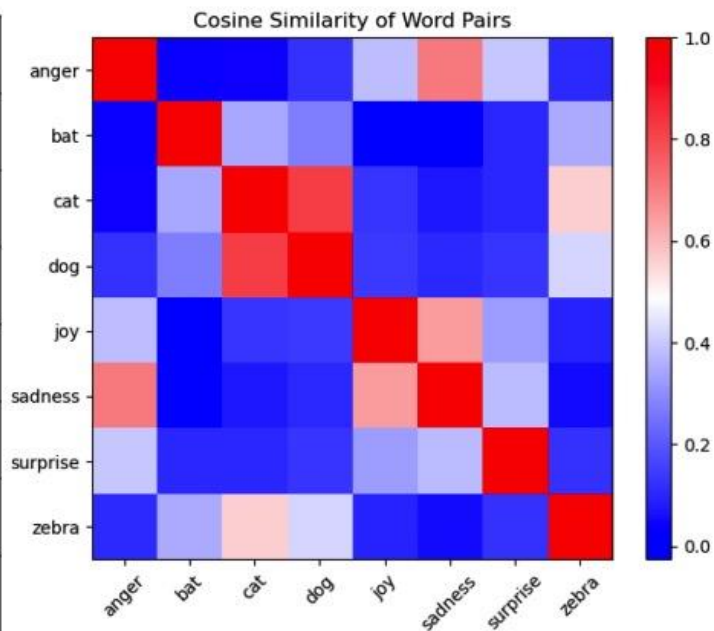
- represent each word in your vocabulary as a vector
 - store as a $V \times D$ matrix
- where:
 V = number of words in vocab.
 D = vector's dimension

Modeling:

- define a model in which the vectors are parameters
- each copy of the word uses the same parameter vector
- train model so that similar words have high cosine similarity

	W				
aardvark	-2.3	0.0	-2.8	...	-4.5
anger	-2.8	-0.9	-1.7	...	-4.3
bat	-4.5	-1.3	0.6	...	-1.7
cat	3.5	-2.0	-2.3	...	-0.4
...				...	
joy	3.0	-0.6	-0.6	...	4.9
...				...	
zebra	-4.7	-4.2	-4.5	...	4.3

in a real use case, the typical embedding dimension is in the hundreds, e.g. $D = 300$



we can't visualise 300 dimensional vectors, but we can inspect their pairwise cosine similarities

Word Embeddings

Q: Why not use one-hot vectors?

A: Too sparse and cannot encode similarity.

Key Idea:

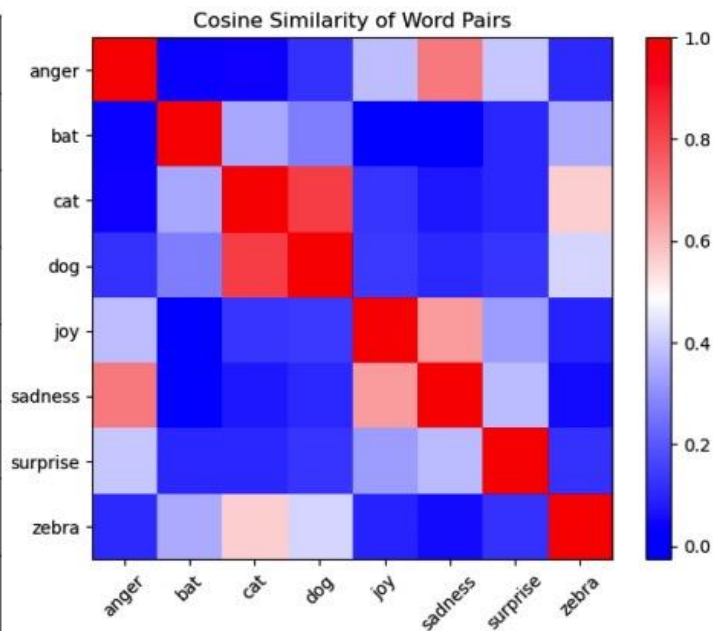
- represent each word in your vocabulary as a vector
 - store as a $V \times D$ matrix
- where:
 V = number of words in vocab.
 D = vector's dimension

Modeling:

- define a model in which the vectors are parameters
- each copy of the word uses the same parameter vector
- train model so that similar words have high cosine similarity

	W				
aardvark	-2.3	0.0	-2.8	...	-4.5
anger	-2.8	-0.9	-1.7	...	-4.3
bat	-4.5	-1.3	0.6	...	-1.7
cat	3.5	-2.0	-2.3	...	-0.4
...				...	
joy	3.0	-0.6	-0.6	...	4.9
...				...	
zebra	-4.7	-4.2	-4.5	...	4.3

in a real use case, the typical embedding dimension is in the hundreds, e.g. $D = 300$

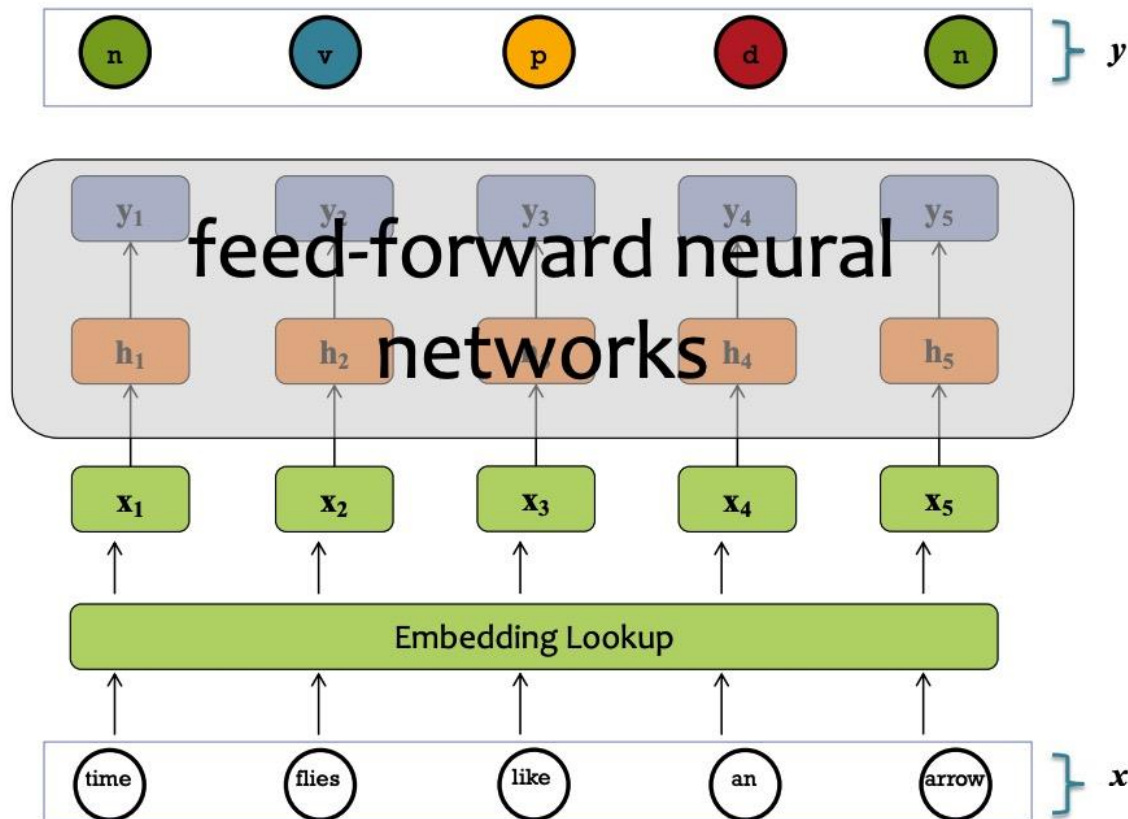


we can't visualise 300 dimensional vectors, but we can inspect their pairwise cosine similarities

Word Embeddings

In all the models we're about to consider (neural networks, RNNs, Transformers) that work with sentences...

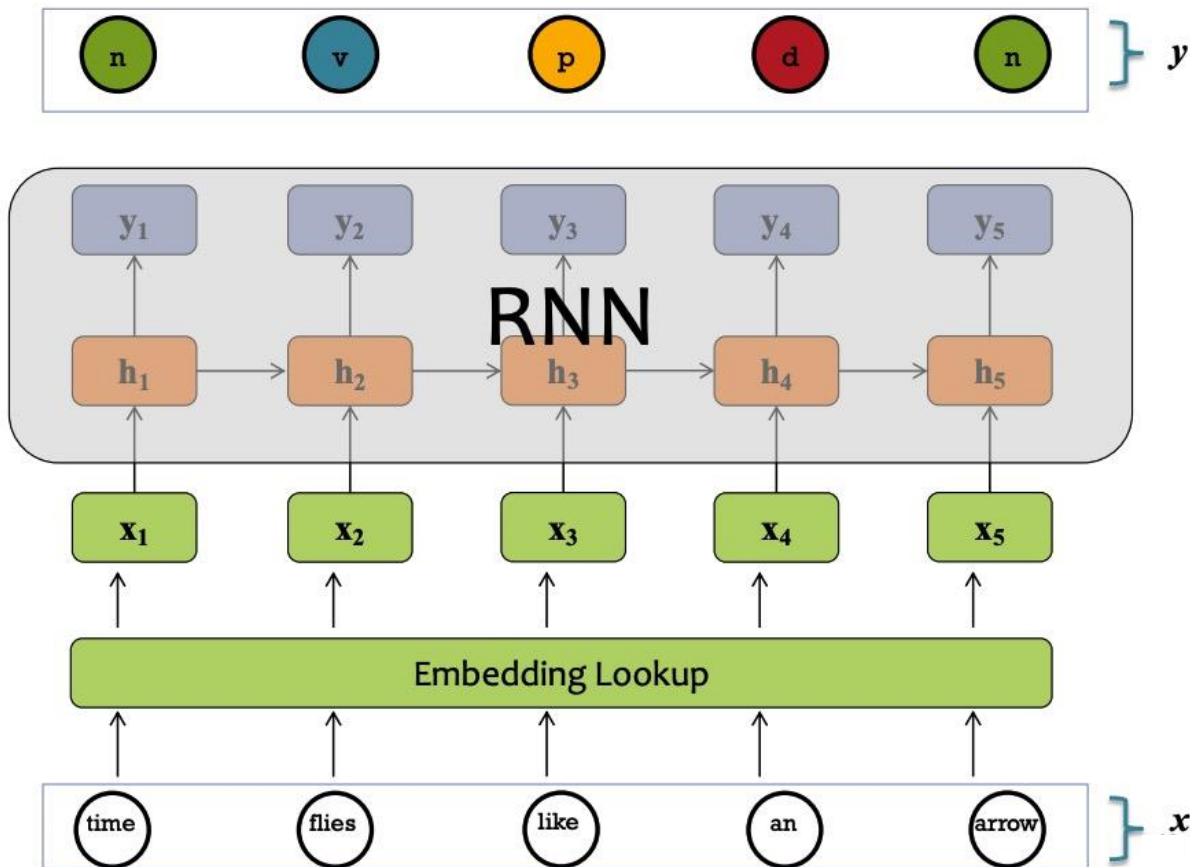
... the first step is to look up the embedding vector for each word at position t and feed it into the model.



Word Embeddings

In all the models we're about to consider (neural networks, RNNs, Transformers) that work with sentences...

... the first step is to look up the embedding vector for each word at position t and feed it into the model.



Part-of-Speech (POS) Tagging

Data: $\mathcal{D} = \{\mathbf{x}^{(n)}, \mathbf{y}^{(n)}\}_{n=1}^N$

Part-of-speech tagging is a classic sequence labelling task. Input is a sentence; output is a sequence of POS tags, same length.

	Noun	Verb	Preposition	Article	Noun	
Sample 1:	n time	v flies	p like	d an	n arrow	$y^{(1)}$ $x^{(1)}$
Sample 2:	n time	n flies	v like	d an	n arrow	$y^{(2)}$ $x^{(2)}$
Sample 3:	n flies	v fly	p with	n their	n wings	$y^{(3)}$ $x^{(3)}$
Sample 4:	p with	n time	n you	v will	v see	$y^{(4)}$ $x^{(4)}$

Q: Why do we need context for POS tagging?

Part-of-Speech (POS) Tagging

Data: $\mathcal{D} = \{\mathbf{x}^{(n)}, \mathbf{y}^{(n)}\}_{n=1}^N$

Part-of-speech tagging is a classic sequence labelling task. Input is a sentence; output is a sequence of POS tags, same length.

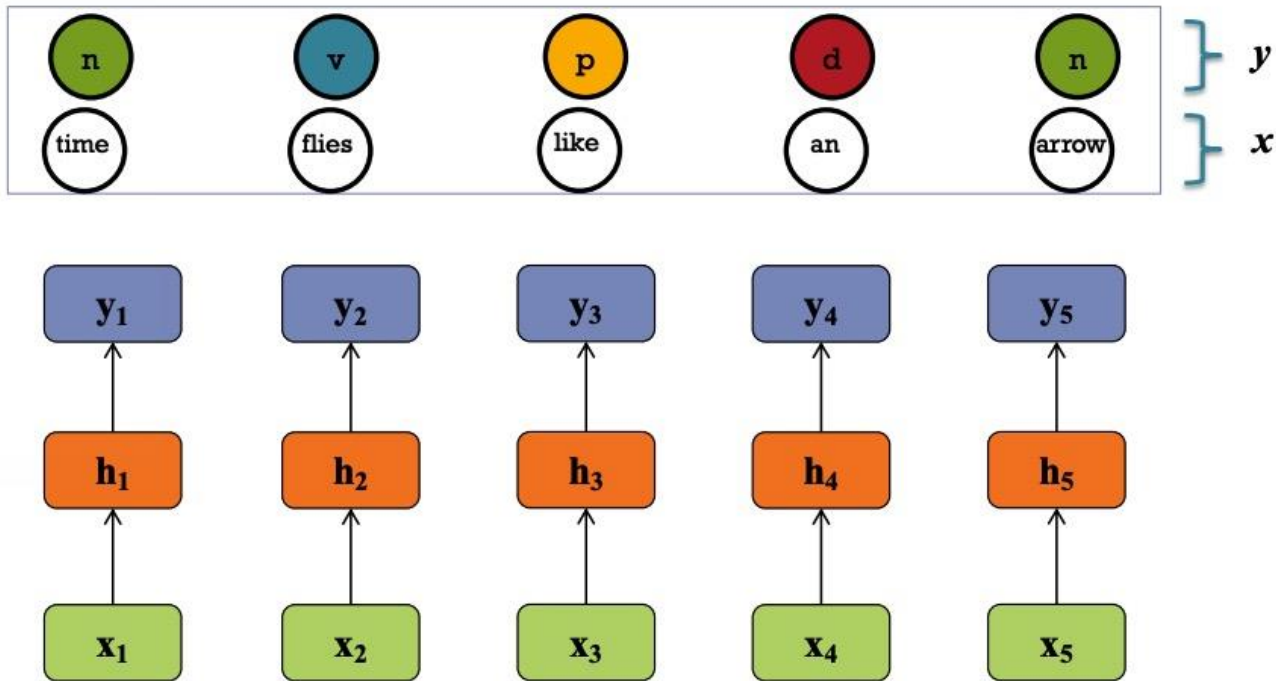
	Noun	Verb	Preposition	Article	Noun	
Sample 1:	n time	v flies	p like	d an	n arrow	} $y^{(1)}$ } $x^{(1)}$
Sample 2:	n time	n flies	v like	d an	n arrow	} $y^{(2)}$ } $x^{(2)}$
Sample 3:	n flies	v fly	p with	n their	n wings	} $y^{(3)}$ } $x^{(3)}$
Sample 4:	p with	n time	n you	v will	v see	} $y^{(4)}$ } $x^{(4)}$

Q: Why do we need context for POS tagging?

A: The tag depends on neighbouring words.

Part-of-Speech (POS) Tagging

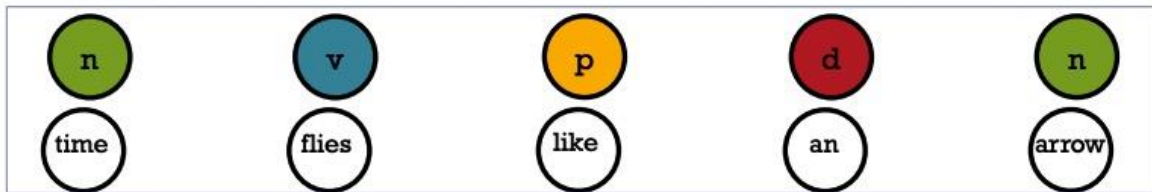
Can we use previous MLP to predict the POS one by one?



Part-of-Speech (POS) Tagging

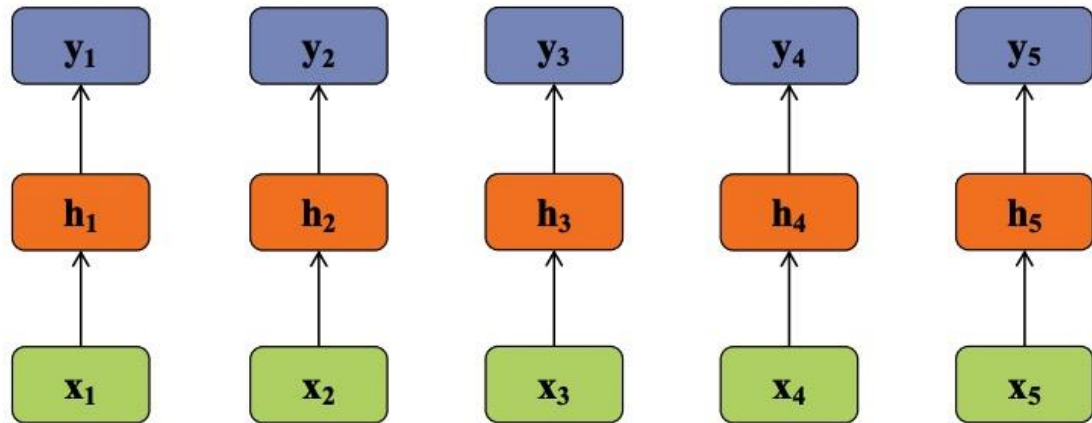
Can we use previous MLP to predict the POS one by one?

Preposition or Verb?



} y
} x

A standard feedforward network processes each word independently; it cannot use information from previous words. For example, 'flies' can be a verb or noun—context decides.

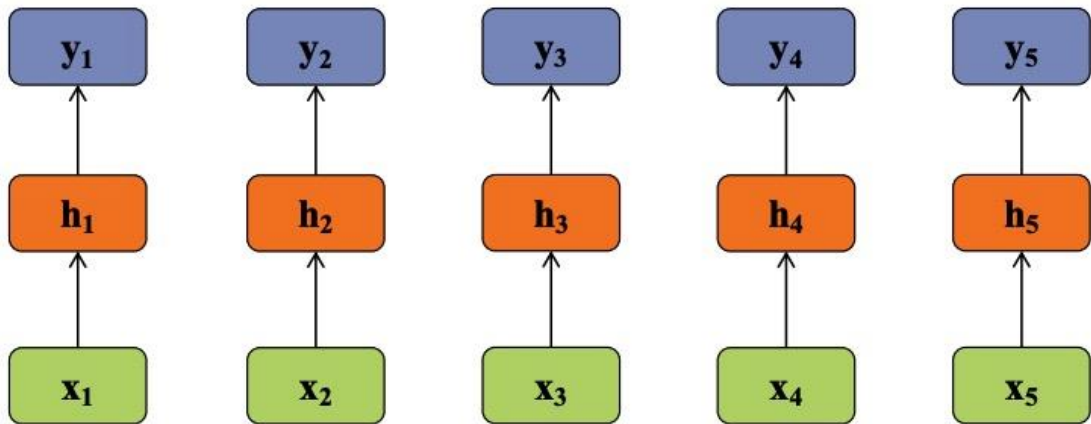
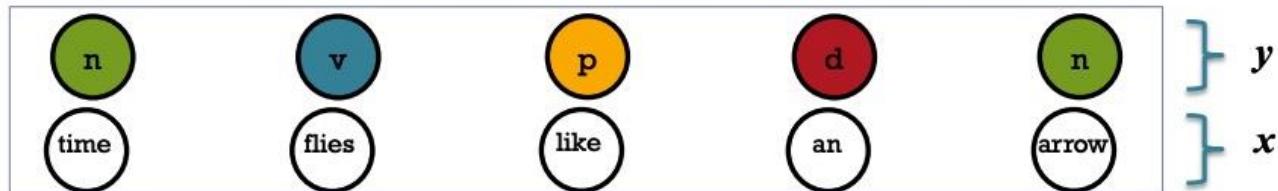


Q: Can we just concatenate several words and feed into an MLP?

Part-of-Speech (POS) Tagging

Can we use previous MLP to predict the POS one by one?

Preposition or Verb?

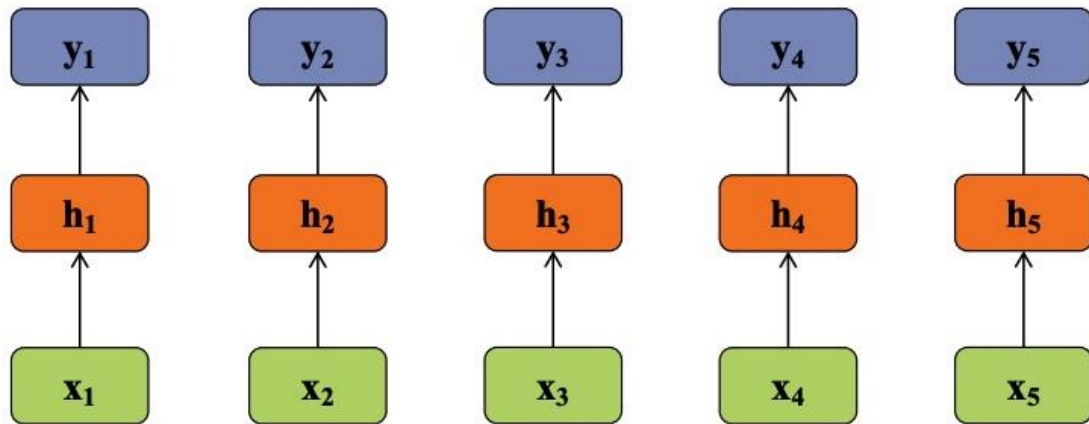
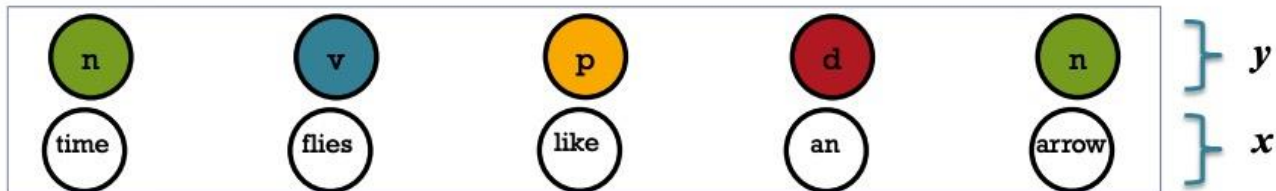


Q: Can we just concatenate several words and feed into an MLP?

A: Not scalable, fixed window, cannot handle variable length.

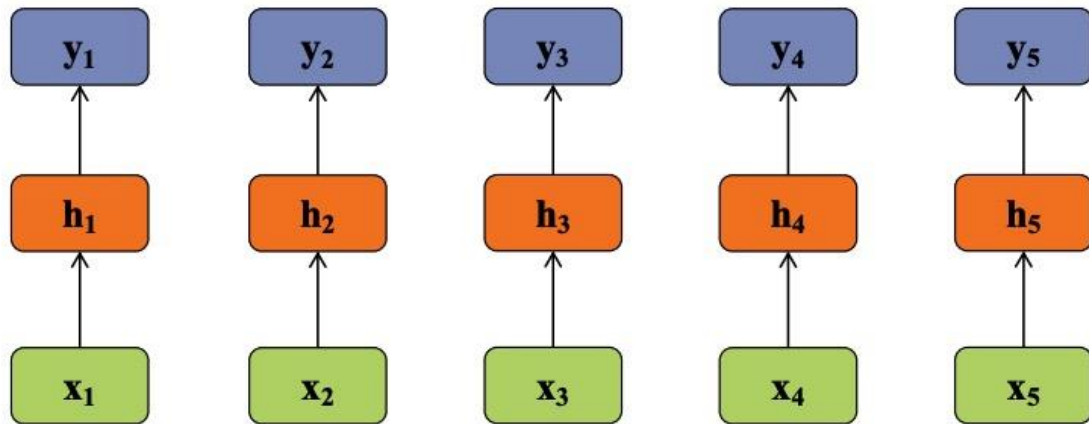
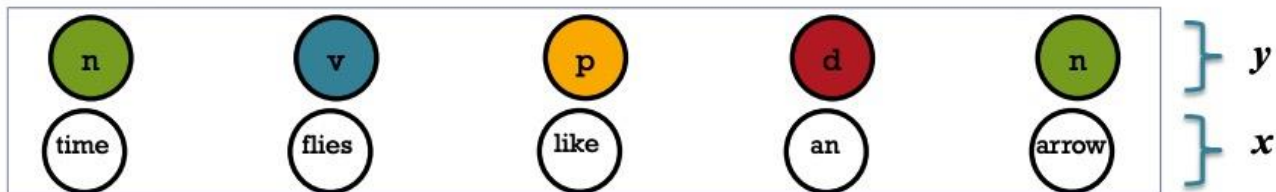
Part-of-Speech (POS) Tagging

1. How could we incorporate **context** (e.g. words to the left/right, or tags to the left/right) into our solution?



Part-of-Speech (POS) Tagging

2. How could we apply the neural networks to a prediction task with **variable length input/output**?



Recurrent Neural Networks (RNNs)

inputs: $\mathbf{x} = (x_1, x_2, \dots, x_T), x_i \in \mathcal{R}^I$
hidden units: $\mathbf{h} = (h_1, h_2, \dots, h_T), h_i \in \mathcal{R}^J$
outputs: $\mathbf{y} = (y_1, y_2, \dots, y_T), y_i \in \mathcal{R}^K$
nonlinearity: $\mathcal{H}$

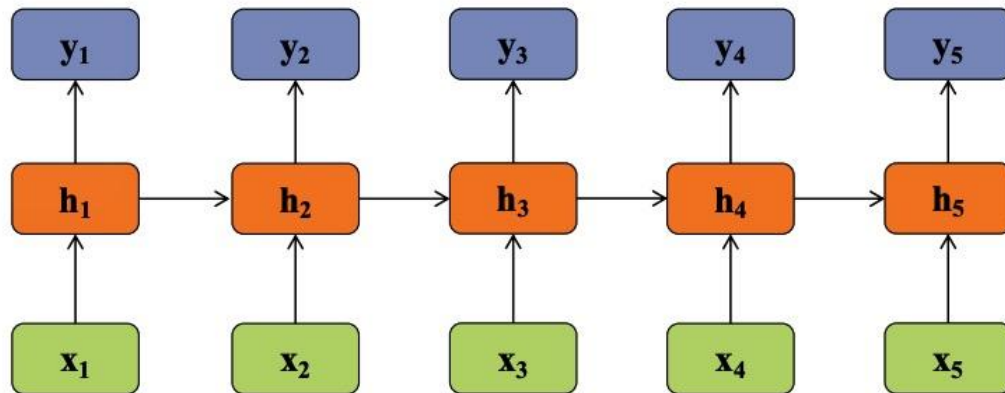
Definition of the RNN:

$$h_t = \mathcal{H}(W_{xh}x_t + W_{hh}h_{t-1} + b_h)$$

$$y_t = W_{hy}h_t + b_y$$

An RNN is a neural network designed to process sequences. It maintains a *hidden state* that carries information from past timesteps.

Q: What does the hidden state represent?



Recurrent Neural Networks (RNNs)

inputs: $\mathbf{x} = (x_1, x_2, \dots, x_T), x_i \in \mathcal{R}^I$
hidden units: $\mathbf{h} = (h_1, h_2, \dots, h_T), h_i \in \mathcal{R}^J$
outputs: $\mathbf{y} = (y_1, y_2, \dots, y_T), y_i \in \mathcal{R}^K$
nonlinearity: $\mathcal{H}$

Definition of the RNN:

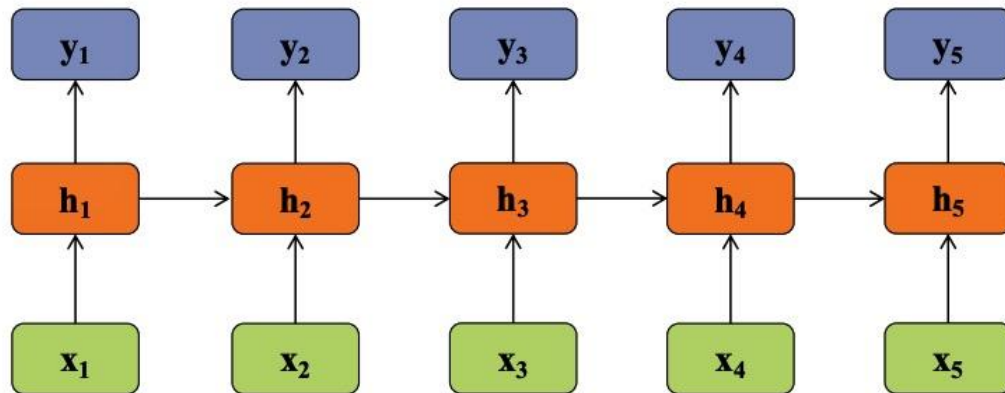
$$h_t = \mathcal{H}(W_{xh}x_t + W_{hh}h_{t-1} + b_h)$$

$$y_t = W_{hy}h_t + b_y$$

An RNN is a neural network designed to process sequences. It maintains a *hidden state* that carries information from past timesteps.

Q: What does the hidden state represent?

A: A compressed memory of past information.



RNNs

inputs: $\mathbf{x} = (x_1, x_2, \dots, x_T), x_i \in \mathcal{R}^I$
hidden units: $\mathbf{h} = (h_1, h_2, \dots, h_T), h_i \in \mathcal{R}^J$
outputs: $\mathbf{y} = (y_1, y_2, \dots, y_T), y_i \in \mathcal{R}^K$
nonlinearity: $\mathcal{H}$

Definition of the RNN:

$$h_t = \mathcal{H}(W_{xh}x_t + W_{hh}h_{t-1} + b_h)$$

$$y_t = W_{hy}h_t + b_y$$

Q: Why do we reuse parameters at every timestep

- If $T=1$, then we have a standard feed-forward neural net with one hidden layer, which requires fixed size inputs/outputs
- By contrast, an RNN can handle arbitrary length inputs/outputs because T can vary from example to example
- The key idea is that we reuse the same parameters at every timestep, always building off of the previous hidden state

RNNs

inputs: $\mathbf{x} = (x_1, x_2, \dots, x_T), x_i \in \mathcal{R}^I$
hidden units: $\mathbf{h} = (h_1, h_2, \dots, h_T), h_i \in \mathcal{R}^J$
outputs: $\mathbf{y} = (y_1, y_2, \dots, y_T), y_i \in \mathcal{R}^K$
nonlinearity: $\mathcal{H}$

Definition of the RNN:

$$h_t = \mathcal{H}(W_{xh}x_t + W_{hh}h_{t-1} + b_h)$$

$$y_t = W_{hy}h_t + b_y$$

Q: Why do we reuse parameters at every timestep?

A: To generalise to any sequence length; also reduces number of parameters.

- If $T=1$, then we have a standard feed-forward neural net with one hidden layer, which requires fixed size inputs/outputs
- By contrast, an RNN can handle arbitrary length inputs/outputs because T can vary from example to example
- The key idea is that we reuse the same parameters at every timestep, always building off of the previous hidden state

RNNs

inputs: $\mathbf{x} = (x_1, x_2, \dots, x_T), x_i \in \mathcal{R}^I$
hidden units: $\mathbf{h} = (h_1, h_2, \dots, h_T), h_i \in \mathcal{R}^J$
outputs: $\mathbf{y} = (y_1, y_2, \dots, y_T), y_i \in \mathcal{R}^K$
nonlinearity: $\mathcal{H}$

Definition of the RNN:

$$h_t = \mathcal{H}(W_{xh}x_t + W_{hh}h_{t-1} + b_h)$$

$$y_t = W_{hy}h_t + b_y$$

Let the input sequence be:

$$x_1 = 1, \quad x_2 = 2, \quad x_3 = 3$$

Parameter	Value
W_{xh}	1
W_{hh}	0.5
b_h	0
W_{hy}	2
b_y	0

RNNs

inputs: $\mathbf{x} = (x_1, x_2, \dots, x_T), x_i \in \mathcal{R}^I$
hidden units: $\mathbf{h} = (h_1, h_2, \dots, h_T), h_i \in \mathcal{R}^J$
outputs: $\mathbf{y} = (y_1, y_2, \dots, y_T), y_i \in \mathcal{R}^K$
nonlinearity: $\mathcal{H}$

Definition of the RNN:

$$h_t = \mathcal{H}(W_{xh}x_t + W_{hh}h_{t-1} + b_h)$$

$$y_t = W_{hy}h_t + b_y$$

Step t = 1

$$h_1 = \text{ReLU}(1 \cdot 1 + 0.5 \cdot 0) = \text{ReLU}(1) = 1$$

$$y_1 = 2 \cdot 1 = 2$$

RNNs

inputs: $\mathbf{x} = (x_1, x_2, \dots, x_T), x_i \in \mathcal{R}^I$
hidden units: $\mathbf{h} = (h_1, h_2, \dots, h_T), h_i \in \mathcal{R}^J$
outputs: $\mathbf{y} = (y_1, y_2, \dots, y_T), y_i \in \mathcal{R}^K$
nonlinearity: $\mathcal{H}$

Definition of the RNN:

$$h_t = \mathcal{H}(W_{xh}x_t + W_{hh}h_{t-1} + b_h)$$

$$y_t = W_{hy}h_t + b_y$$

Step t = 2

$$h_2 = \text{ReLU}(1 \cdot 2 + 0.5 \cdot 1) = \text{ReLU}(2 + 0.5) = \text{ReLU}(2.5) = 2.5$$

$$y_2 = 2 \cdot 2.5 = 5$$

RNNs

inputs: $\mathbf{x} = (x_1, x_2, \dots, x_T), x_i \in \mathcal{R}^I$
hidden units: $\mathbf{h} = (h_1, h_2, \dots, h_T), h_i \in \mathcal{R}^J$
outputs: $\mathbf{y} = (y_1, y_2, \dots, y_T), y_i \in \mathcal{R}^K$
nonlinearity: $\mathcal{H}$

Definition of the RNN:

$$h_t = \mathcal{H}(W_{xh}x_t + W_{hh}h_{t-1} + b_h)$$

$$y_t = W_{hy}h_t + b_y$$

Step t = 3

$$h_3 = \text{ReLU}(1 \cdot 3 + 0.5 \cdot 2.5) = \text{ReLU}(3 + 1.25) = \text{ReLU}(4.25) = 4.25$$

$$y_3 = 2 \cdot 4.25 = 8.5$$

Final RNN outputs (ReLU)

$$h^{\rightarrow} = [1, 2.5, 4.25]$$

$$y^{\rightarrow} = [2, 5, 8.5]$$

Bidirectional RNN

inputs: $\mathbf{x} = (x_1, x_2, \dots, x_T), x_i \in \mathcal{R}^I$
hidden units: $\vec{\mathbf{h}}$ and $\overleftarrow{\mathbf{h}}$
outputs: $\mathbf{y} = (y_1, y_2, \dots, y_T), y_i \in \mathcal{R}^K$
nonlinearity: $\mathcal{H}$

Recursive Definition:

$$\vec{h}_t = \mathcal{H} \left(W_{x\vec{h}} x_t + W_{\vec{h}\vec{h}} \vec{h}_{t-1} + b_{\vec{h}} \right)$$

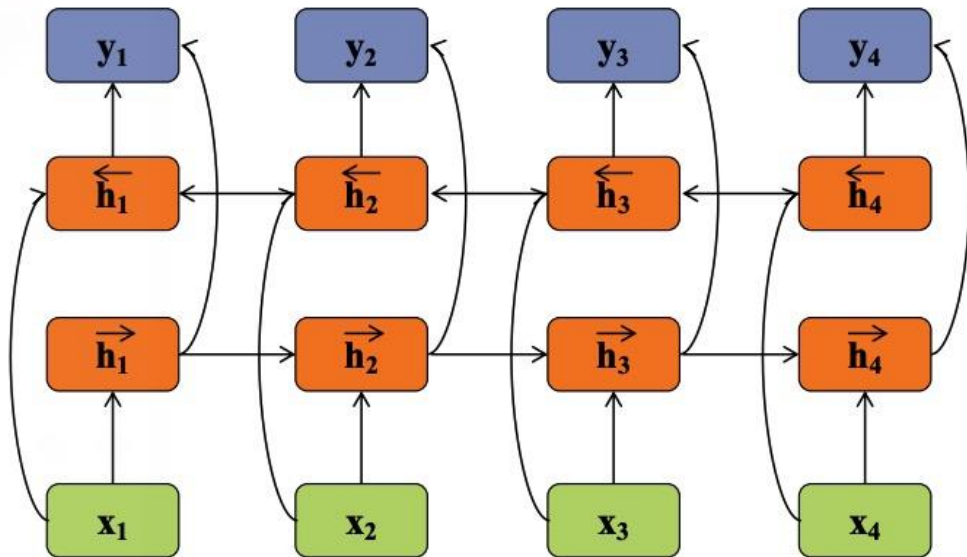
$$\overleftarrow{h}_t = \mathcal{H} \left(W_{x\overleftarrow{h}} x_t + W_{\overleftarrow{h}\overleftarrow{h}} \overleftarrow{h}_{t+1} + b_{\overleftarrow{h}} \right)$$

$$y_t = W_{\vec{h}y} \vec{h}_t + W_{\overleftarrow{h}y} \overleftarrow{h}_t + b_y$$

A Bidirectional RNN processes the sequence twice:

- Forward from left to right
- Backward from right to left

Q: Why do we need backward context?



Bidirectional RNN

inputs: $\mathbf{x} = (x_1, x_2, \dots, x_T), x_i \in \mathcal{R}^I$
hidden units: $\vec{\mathbf{h}}$ and $\overleftarrow{\mathbf{h}}$
outputs: $\mathbf{y} = (y_1, y_2, \dots, y_T), y_i \in \mathcal{R}^K$
nonlinearity: $\mathcal{H}$

Recursive Definition:

$$\vec{h}_t = \mathcal{H} \left(W_{x\vec{h}} x_t + W_{\vec{h}\vec{h}} \vec{h}_{t-1} + b_{\vec{h}} \right)$$

$$\overleftarrow{h}_t = \mathcal{H} \left(W_{x\overleftarrow{h}} x_t + W_{\overleftarrow{h}\overleftarrow{h}} \overleftarrow{h}_{t+1} + b_{\overleftarrow{h}} \right)$$

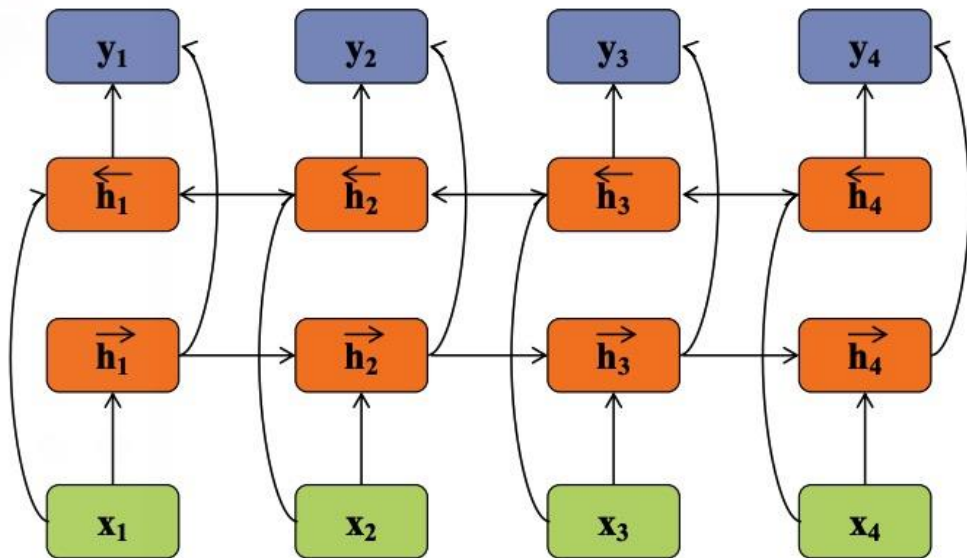
$$y_t = W_{\vec{h}y} \vec{h}_t + W_{\overleftarrow{h}y} \overleftarrow{h}_t + b_y$$

A Bidirectional RNN processes the sequence twice:

- Forward from left to right
- Backward from right to left

Q: Why do we need backward context?

A: Some tasks depend on words ahead



Bidirectional RNN

inputs: $\mathbf{x} = (x_1, x_2, \dots, x_T), x_i \in \mathcal{R}^I$
hidden units: $\vec{\mathbf{h}}$ and $\overleftarrow{\mathbf{h}}$
outputs: $\mathbf{y} = (y_1, y_2, \dots, y_T), y_i \in \mathcal{R}^K$
nonlinearity: $\mathcal{H}$

Recursive Definition:

$$\vec{h}_t = \mathcal{H} \left(W_{x\vec{h}} x_t + W_{\vec{h}\vec{h}} \vec{h}_{t-1} + b_{\vec{h}} \right)$$

$$\overleftarrow{h}_t = \mathcal{H} \left(W_{x\overleftarrow{h}} x_t + W_{\overleftarrow{h}\overleftarrow{h}} \overleftarrow{h}_{t+1} + b_{\overleftarrow{h}} \right)$$

$$y_t = W_{\vec{h}y} \vec{h}_t + W_{\overleftarrow{h}y} \overleftarrow{h}_t + b_y$$

Inputs: $x = (x_1, x_2, x_3) = (1, 2, 3)$

$$\vec{h}_0 = 0$$

$$\overleftarrow{h}_4 = 0 \text{ (since sequence length } T = 3)$$

Forward:

- $W_{x\vec{h}} = 1$
- $W_{\vec{h}\vec{h}} = 0.5$
- $b_{\vec{h}} = 0$

Output:

- $W_{\vec{h}y} = 1$
- $W_{\overleftarrow{h}y} = 1$
- $b_y = 0$

Backward:

- $W_{x\overleftarrow{h}} = 1$
- $W_{\overleftarrow{h}\overleftarrow{h}} = 0.5$
- $b_{\overleftarrow{h}} = 0$

Bidirectional RNN

inputs: $\mathbf{x} = (x_1, x_2, \dots, x_T), x_i \in \mathcal{R}^I$
hidden units: $\vec{\mathbf{h}}$ and $\overleftarrow{\mathbf{h}}$
outputs: $\mathbf{y} = (y_1, y_2, \dots, y_T), y_i \in \mathcal{R}^K$
nonlinearity: $\mathcal{H}$

Recursive Definition:

$$\vec{h}_t = \mathcal{H} \left(W_{x\vec{h}} x_t + W_{\vec{h}\vec{h}} \vec{h}_{t-1} + b_{\vec{h}} \right)$$

$$\overleftarrow{h}_t = \mathcal{H} \left(W_{x\overleftarrow{h}} x_t + W_{\overleftarrow{h}\overleftarrow{h}} \overleftarrow{h}_{t+1} + b_{\overleftarrow{h}} \right)$$

$$y_t = W_{\vec{h}y} \vec{h}_t + W_{\overleftarrow{h}y} \overleftarrow{h}_t + b_y$$

Forward

$$\vec{h}_t = \text{ReLU}(1 \cdot x_t + 0.5 \cdot \vec{h}_{t-1})$$

- $t=1$

$$\vec{h}_1 = \text{ReLU}(1 \cdot 1 + 0.5 \cdot 0) = \text{ReLU}(1) = 1$$

- $t=2$

$$\vec{h}_2 = \text{ReLU}(1 \cdot 2 + 0.5 \cdot 1) = \text{ReLU}(2 + 0.5) = 2.5$$

- $t=3$

$$\vec{h}_3 = \text{ReLU}(1 \cdot 3 + 0.5 \cdot 2.5) = \text{ReLU}(3 + 1.25) = 4.25$$

So:

$$\vec{h} = (1, 2.5, 4.25)$$

Bidirectional RNN

inputs: $\mathbf{x} = (x_1, x_2, \dots, x_T), x_i \in \mathcal{R}^I$
hidden units: $\vec{\mathbf{h}}$ and $\overleftarrow{\mathbf{h}}$
outputs: $\mathbf{y} = (y_1, y_2, \dots, y_T), y_i \in \mathcal{R}^K$
nonlinearity: $\mathcal{H}$

Recursive Definition:

$$\vec{h}_t = \mathcal{H} \left(W_{x\vec{h}} x_t + W_{\vec{h}\vec{h}} \vec{h}_{t-1} + b_{\vec{h}} \right)$$

$$\overleftarrow{h}_t = \mathcal{H} \left(W_{x\overleftarrow{h}} x_t + W_{\overleftarrow{h}\overleftarrow{h}} \overleftarrow{h}_{t+1} + b_{\overleftarrow{h}} \right)$$

$$y_t = W_{\vec{h}y} \vec{h}_t + W_{\overleftarrow{h}y} \overleftarrow{h}_t + b_y$$

Backward

$$\overleftarrow{h}_t = \text{ReLU}(1 \cdot x_t + 0.5 \cdot \overleftarrow{h}_{t+1})$$

- $t=3$

$$\overleftarrow{h}_3 = \text{ReLU}(1 \cdot 3 + 0.5 \cdot 0) = \text{ReLU}(3) = 3$$

- $t=2$

$$\overleftarrow{h}_2 = \text{ReLU}(1 \cdot 2 + 0.5 \cdot 3) = \text{ReLU}(2 + 1.5) = 3.5$$

- $t=1$

$$\overleftarrow{h}_1 = \text{ReLU}(1 \cdot 1 + 0.5 \cdot 3.5) = \text{ReLU}(1 + 1.75) = 2.75$$

So:

$$\overleftarrow{\mathbf{h}} = (2.75, 3.5, 3)$$

Bidirectional RNN

inputs: $\mathbf{x} = (x_1, x_2, \dots, x_T), x_i \in \mathcal{R}^I$
hidden units: $\vec{\mathbf{h}}$ and $\overleftarrow{\mathbf{h}}$
outputs: $\mathbf{y} = (y_1, y_2, \dots, y_T), y_i \in \mathcal{R}^K$
nonlinearity: $\mathcal{H}$

Recursive Definition:

$$\vec{h}_t = \mathcal{H} \left(W_{x\vec{h}} x_t + W_{\vec{h}\vec{h}} \vec{h}_{t-1} + b_{\vec{h}} \right)$$

$$\overleftarrow{h}_t = \mathcal{H} \left(W_{x\overleftarrow{h}} x_t + W_{\overleftarrow{h}\overleftarrow{h}} \overleftarrow{h}_{t+1} + b_{\overleftarrow{h}} \right)$$

$$y_t = W_{\vec{h}y} \vec{h}_t + W_{\overleftarrow{h}y} \overleftarrow{h}_t + b_y$$

Output

$$y_t = 1 \cdot \vec{h}_t + 1 \cdot \overleftarrow{h}_t + 0 = \vec{h}_t + \overleftarrow{h}_t$$

$$y_1 = 1 + 2.75 = 3.75$$

$$y_2 = 2.5 + 3.5 = 6$$

$$y_3 = 4.25 + 3 = 7.25$$

Deep RNNs

inputs: $\mathbf{x} = (x_1, x_2, \dots, x_T), x_i \in \mathcal{R}^I$

outputs: $\mathbf{y} = (y_1, y_2, \dots, y_T), y_i \in \mathcal{R}^K$

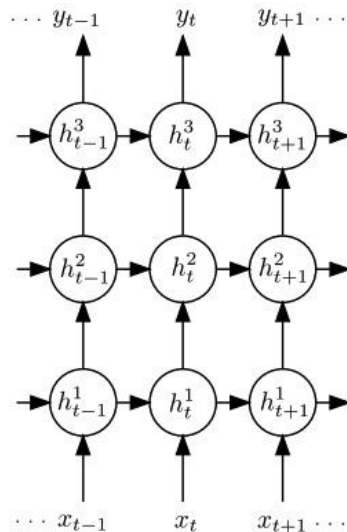
nonlinearity: $\mathcal{H}$

Recursive Definition:

$$h_t^n = \mathcal{H}(W_{h^{n-1}h^n}h_t^{n-1} + W_{h^nh^n}h_{t-1}^n + b_h^n)$$

$$y_t = W_{h^N y}h_t^N + b_y$$

RNN layers can be stacked to form deep RNNs. Each layer reads the hidden states of the previous one, improving representational power.



Q: What is the main risk of deep RNNs?

Deep RNNs

inputs: $\mathbf{x} = (x_1, x_2, \dots, x_T), x_i \in \mathcal{R}^I$

outputs: $\mathbf{y} = (y_1, y_2, \dots, y_T), y_i \in \mathcal{R}^K$

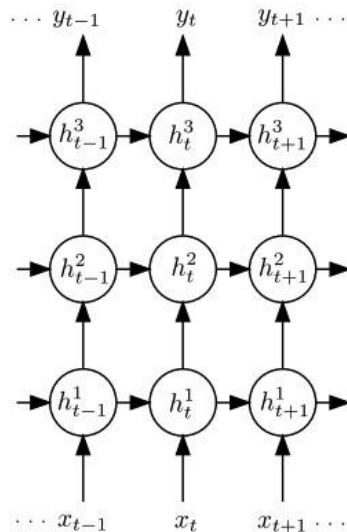
nonlinearity: $\mathcal{H}$

Recursive Definition:

$$h_t^n = \mathcal{H}(W_{h^{n-1}h^n}h_t^{n-1} + W_{h^nh^n}h_{t-1}^n + b_h^n)$$

$$y_t = W_{h^N y}h_t^N + b_y$$

RNN layers can be stacked to form deep RNNs. Each layer reads the hidden states of the previous one, improving representational power.



Q: What is the main risk of deep RNNs?

A: Vanishing gradients.

Implement RNN using PyTorch

```
x = torch.tensor([
    [[1.0, 0.0]], # x1
    [[0.0, 1.0]], # x2
    [[1.0, 1.0]]  # x3
]) # shape: (3, 1, 2)
```

```
rnn = nn.RNN(
    input_size=2,
    hidden_size=5, #
    nonlinearity='relu',
    batch_first=False
)
```

Q: What does `batch_first=True` do?

Implement RNN using PyTorch

```
x = torch.tensor([
    [[1.0, 0.0]], # x1
    [[0.0, 1.0]], # x2
    [[1.0, 1.0]]  # x3
]) # shape: (3, 1, 2)
```

```
rnn = nn.RNN(
    input_size=2,
    hidden_size=5, #
    nonlinearity='relu',
    batch_first=False
)
```

Q: What does `batch_first=True` do?

A: Makes tensor shape (batch, seq_len, input_dim).

Sketch RNN Demo

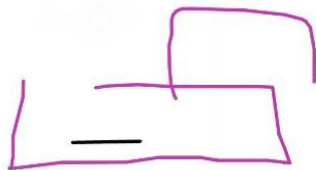
https://magenta.tensorflow.org/assets/sketch_rnn_demo/index.html

 info  random  clear

Model:

start drawing truck.

This demo shows how RNNs can generate sequences — here, drawing strokes step-by-step. RNNs are generative over time.



Summary

Why RNNs?

- Traditional feedforward networks and CNNs cannot handle variable-length sequences or temporal/contextual dependencies.
- Many tasks (POS tagging) require using previous/future words for correct prediction.

Word Embeddings as Inputs

- Each word is represented as a dense vector looked up from a $V \times D$ embedding matrix.
- Embeddings are the first step in any sequence model (RNN, LSTM, GRU, Transformer).

Core Idea of RNNs

- RNNs process inputs one timestep at a time, maintaining a hidden state: Captures information from previous steps.
- Same parameters reused across all timesteps → works for any sequence length.